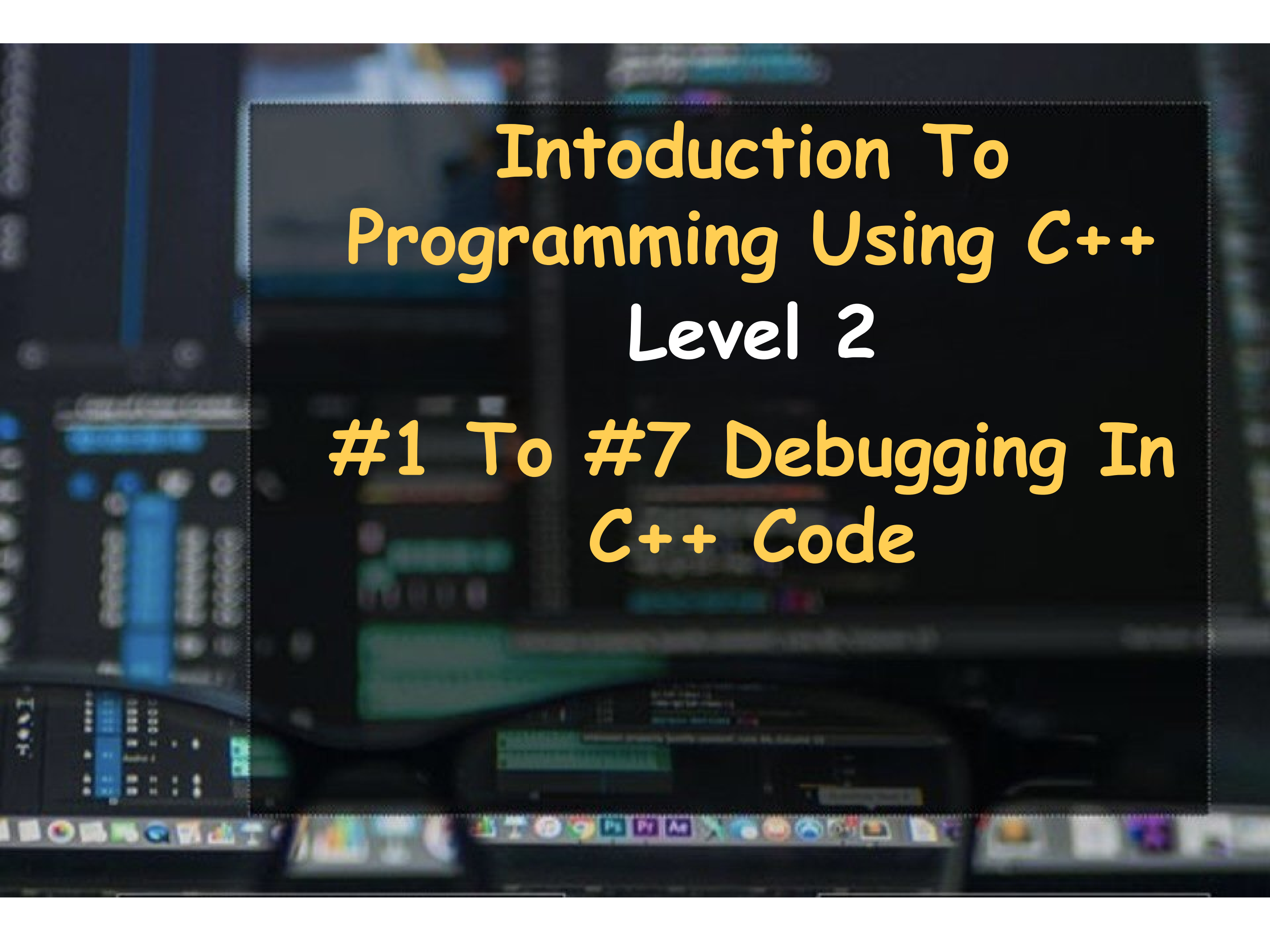
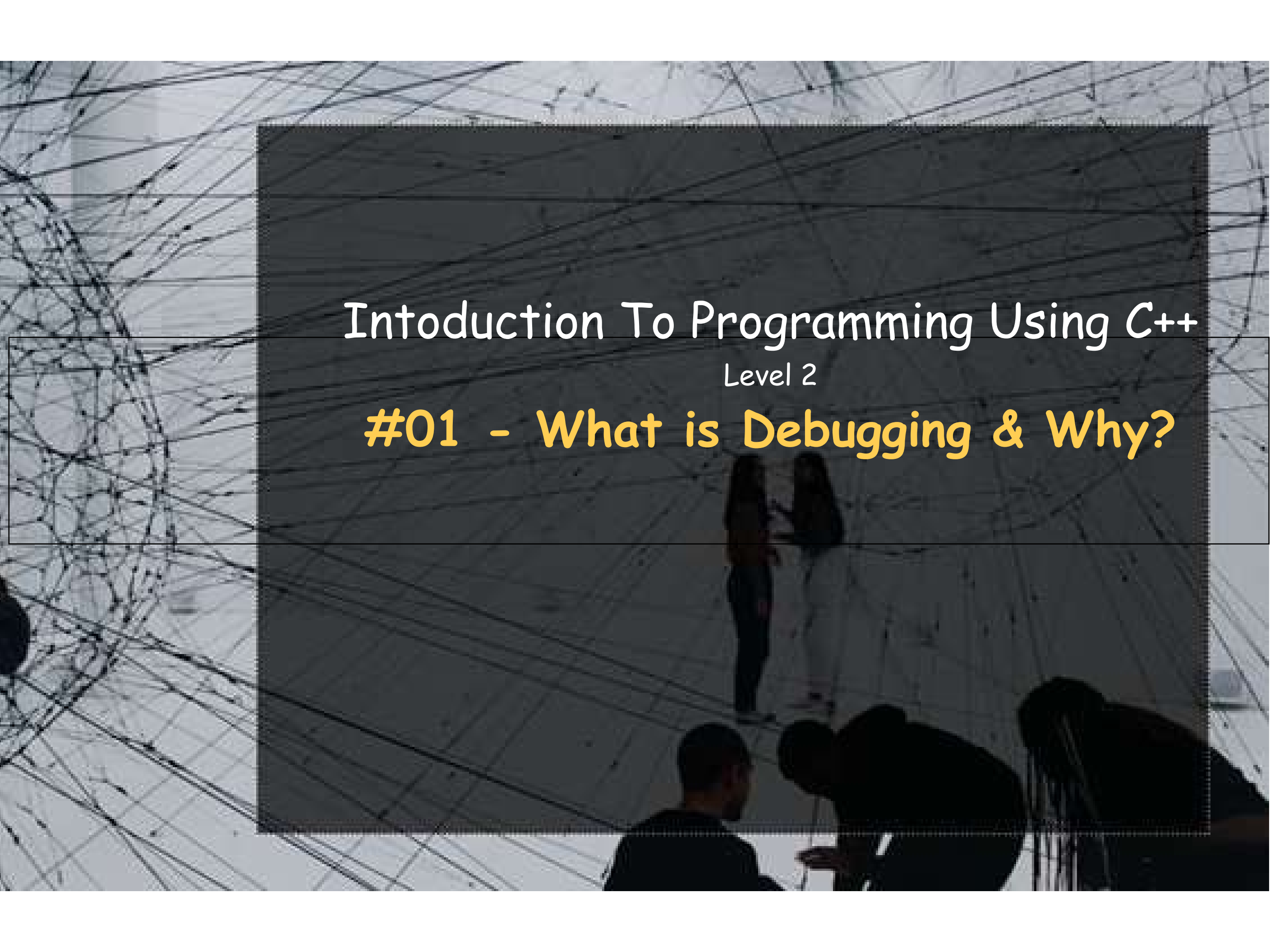


The background is a blurred image of a computer monitor. On the left, a vertical sidebar of a code editor is visible with blue highlights. The main area of the screen shows lines of code in a dark theme. At the bottom, a taskbar with various application icons is visible. Overlaid on this background is a semi-transparent dark rectangle containing the title text.

Introduction To Programming Using C++ Level 2

#1 To #7 Debugging In C++ Code



Introduction To Programming Using C++

Level 2

#01 - What is Debugging & Why?

Definition

- ◆ Debugging is the process of running a program in a controlled way to find, understand, and fix errors in the code.
- ◆ During debugging, we can execute the code step by step, track Variables and Values, and observe how Functions behave.

Explanation

- C++ is a Compiled Language.
- When we Build a C++ program, the source code is compiled and linked to produce an executable file such as .exe.
- The executable can then be Run by the operating system.
- This is different from an Interpreted Language, where code is generally executed by an interpreter rather than being compiled into a standalone executable first.

Procedures

- 1 Write the source code.
- 2 Build the program.
- 3 The compiler checks and translates the source code.
- 4 The Linker combines the required parts and produces the executable.
- 5 Run the executable.

💡 Example

◆ Suppose we have:

```
int x = 10;
```

```
int y = 5;
```

```
int result = x + y;
```

◆ During debugging, we can stop the program and check the value of x, y, and result step by step.

➡ Result

We can understand exactly how the program is executing and identify where the problem occurs.

Note

◆ Debugging is not only about finding syntax errors. It is especially useful for finding problems that happen while the program is running or problems where the program produces the wrong result.

⊘ Warning

◆ A program can compile successfully and still contain Runtime Errors or Logical Errors.



Comparison

▨ Compiled Language

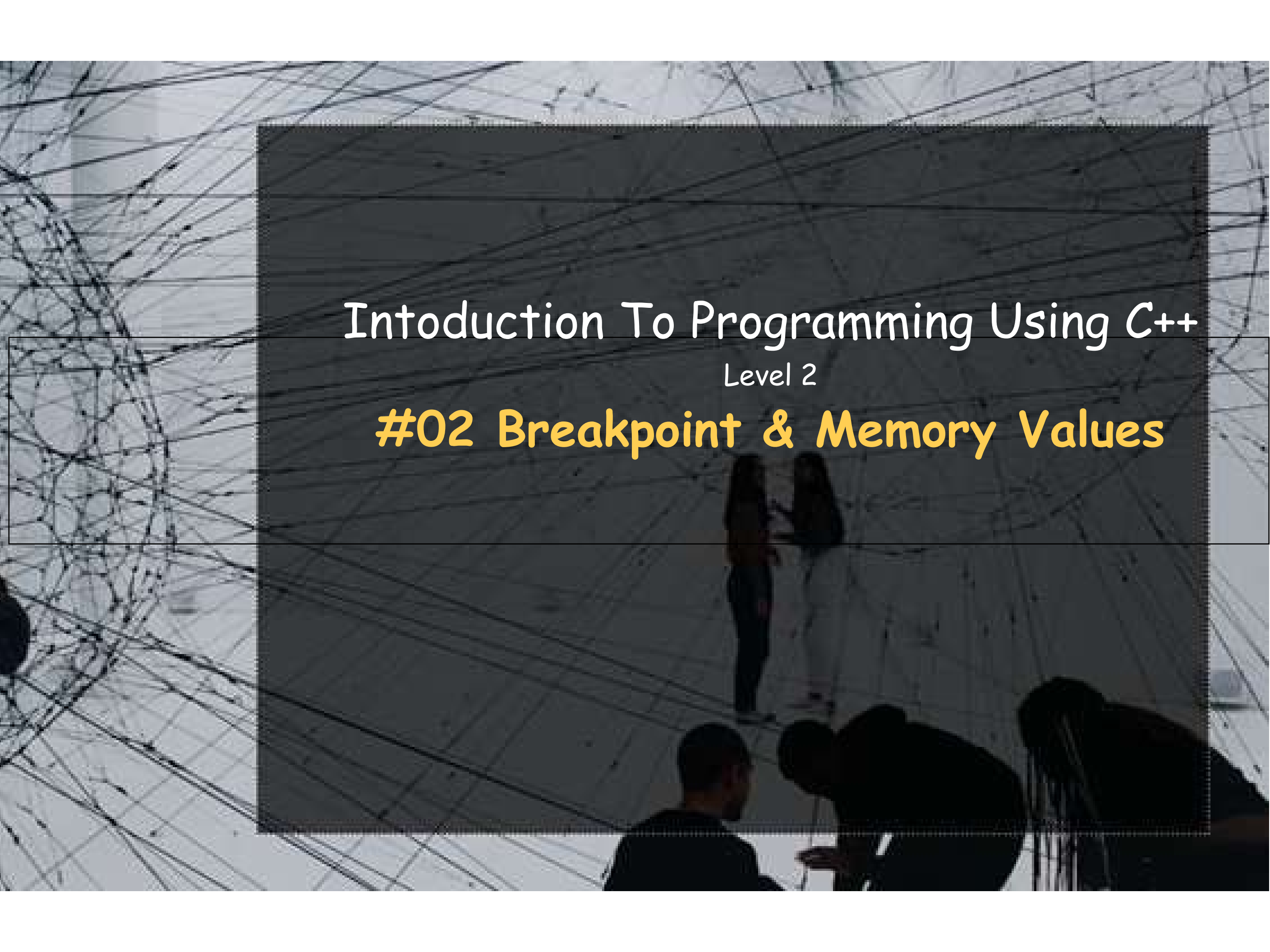
- ✓ The source code is compiled before execution.
- ✓ C++ is primarily a compiled language.
- ✓ The build process can produce an executable file.

▤ Interpreted Language

- ✓ Code is generally executed through an interpreter at runtime.
- ✓ It does not follow the same traditional compile-and-link-to-executable process as C++.
- ✓ "Line by line" is a simplified description and does not apply identically to every interpreted language.

✦✦ Summary

- ✓ Debugging helps us find and fix problems in programs.
- ✓ It allows us to inspect execution, variables, values, and functions.



Introduction To Programming Using C++

Level 2

#02 Breakpoint & Memory Values

Definition

- ◆ A Breakpoint is a point in the code where the debugger pauses program execution.
- ◆ When the program reaches a Breakpoint, execution pauses before the instruction on that line is executed.

Explanation

- A Breakpoint allows us to stop the program at a specific point during debugging.
- When the program stops, we can inspect Variables and their current Values.
- The program is still running under the debugger, but its execution is temporarily paused.
- A Breakpoint should normally be placed on an executable line of code.

Procedures

1. Place a Breakpoint on the desired line.
2. Start Debugging.
3. The program executes normally until it reaches the Breakpoint.
4. Execution pauses at the Breakpoint.
5. Use Step Into (F11) to execute the program step by step.
6. Inspect the current Variables and Values.

💡 Example

◆ Suppose we have:

```
int x = 10;
```

```
int y = 20;
```

```
int result = x + y;
```

◆ If we place a Breakpoint on the result line, the debugger stops there before that statement is executed.

◆ We can then inspect the current values of x and y.

➡ Result

We can inspect the current state of the program before the selected line is executed.

Note

- ◆ Debug Mode provides debugging information and tools that make it easier to investigate the program.
- ◆ Release Mode is normally used for the final or optimized version of the program.

⊘ Warning

◆ Debug builds can behave differently from Release builds because compiler optimizations and debugging information can differ.



Comparison

Debug Mode

- ✓ Designed for debugging and development.
- ✓ Makes it easier to inspect Variables and execution.
- ✓ Usually contains additional debugging information.

Release Mode

- ✓ Designed for the final or production build.
- ✓ Usually uses more compiler optimizations.
- ✓ Focuses more on execution performance.

✦✦ Summary

- ✓ A Breakpoint pauses program execution at a specific line.
- ✓ F11 (Step Into) allows us to execute the program step by step.



Introduction To Programming Using C++

Level 2

#03 More About Breakpoints

Definition

- ◆ Multiple Breakpoints can be placed at different locations in the same program.
- ◆ They allow us to stop execution at specific points that we want to investigate.

Explanation

- When we press F5, the program runs until it reaches the next enabled Breakpoint.
- When we press F5 again, execution continues until the next enabled Breakpoint.
- Breakpoints can be temporarily disabled without deleting them.
- In Visual Studio, we can use Debug → Disable All Breakpoints to temporarily disable all Breakpoints.

Procedures

1. Place a Breakpoint on an executable line.
2. Place additional Breakpoints at other locations if needed.
3. Press F5 to start or continue debugging.
4. The program stops at the next enabled Breakpoint.
5. Press F5 again to continue execution.
6. Enable or disable Breakpoints depending on what you want to investigate.

💡 Example

- ◆ Suppose we have Breakpoints on lines 5, 10, and 15.
- ◆ Press F5 → execution stops at line 5.
- ◆ Press F5 again → execution continues and stops at line 10.
- ◆ Press F5 again → execution continues and stops at line 15.

➡ Result

We can investigate different parts of the program without manually stepping through every line.

Note

◆ A disabled Breakpoint remains configured in the code but does not pause execution until it is enabled again.

⊘ Warning

- ◆ Having too many Breakpoints can make debugging harder to follow.



Comparison

Enabled Breakpoint

- ✓ Pauses execution when the program reaches it.
- ✓ Useful for investigating a specific part of the program.
- ✓ Can be reached by pressing F5.

Disabled Breakpoint

- ✓ Remains configured in the code.
- ✓ Does not pause execution.
- ✓ Can be enabled again later.

✦✦ Summary

- ✓ Multiple Breakpoints allow us to investigate different locations.
- ✓ F5 continues execution until the next enabled Breakpoint.

Introduction To Programming Using C++

Level 2

#04 Autos Window



Definition

- ◆ The Autos Window displays relevant Variables and their current Values while debugging.
- ◆ It automatically shows information related to the current execution point.

Explanation

- The Autos Window automatically displays Variables that are relevant to the current line of execution.
- It updates as we move through the program step by step.
- It is useful for quickly checking the current state of Variables.
- We do not need to manually add every Variable that we want to inspect.

Procedures

1. Start Debugging.
2. Stop the program at a Breakpoint.
3. Open the Autos Window.
4. Observe the displayed Variables and Values.
5. Use Step Into or another stepping command.
6. Observe how the Values change during execution.

💡 Example

◆ Suppose we have:

```
int x = 10;  
x = x + 5;
```

◆ When debugging, the Autos Window can show $x = 10$ before the assignment is executed.

◆ After executing the assignment, it can show $x = 15$.

➡ Result

We can observe how Variable Values change during program execution.

Note

◆ The Autos Window is automatically populated by the debugger based on the current execution context.

⊘ Warning

◆ A Variable may not appear in the Autos Window if it is not relevant to the current execution point.

✦✦ Summary

- ✓ The Autos Window automatically displays relevant Variables.
- ✓ It helps us monitor Values while stepping through the program.



Introduction To Programming Using C++

Level 2

#05 Quick Watch Window

Definition

- ◆ Quick Watch is a debugging feature that allows us to inspect a selected Variable or Expression manually.
- ◆ It is useful when we want to investigate a specific value or calculation.

Explanation

- Unlike the Autos Window, Quick Watch does not simply display Variables automatically.
- We select the Variable or Expression that we want to investigate.
- Quick Watch can evaluate Expressions and show their current Values.
- It can be used to check a Variable, part of an Expression, or the result of a calculation.

Procedures

1. Start Debugging.
2. Stop the program at a Breakpoint.
3. Select a Variable or Expression.
4. Press Shift + F9 to open Quick Watch.
5. Inspect the current Value or Expression result.
6. Continue debugging when finished.

💡 Example

◆ Suppose we have:

```
int x = 10;
```

```
int y = 20;
```

```
int result = x + y;
```

◆ Select `x + y` and open Quick Watch.

◆ The debugger evaluates the Expression and shows the current result.

➡ Result

We can quickly investigate a specific Expression without adding it permanently to a Watch Window.

Note

◆ An Array or Object address may be displayed in Hexadecimal notation because memory addresses are commonly represented in Hexadecimal.

⊘ Warning

◆ The Value shown by Quick Watch depends on the current debugging state and execution point.



Comparison

Autos Window

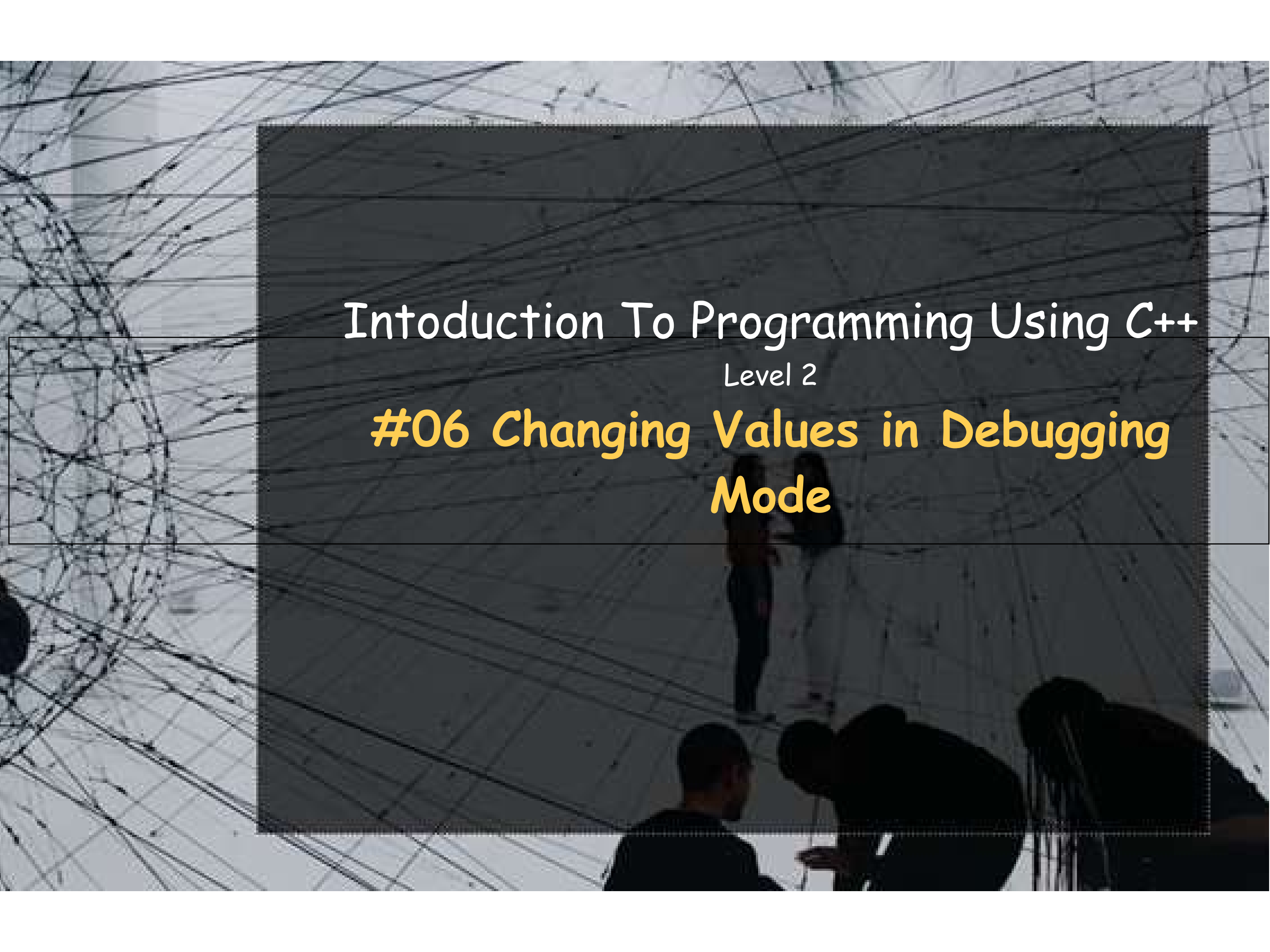
- ✓ Automatically displays relevant Variables.
- ✓ Changes as execution moves.
- ✓ Useful for quickly observing the current program state.

Quick Watch

- ✓ Requires us to select what we want to inspect.
- ✓ Can evaluate specific Expressions.
- ✓ Useful for investigating one particular Value or calculation.

✦✦ Summary

- ✓ Autos shows relevant Values automatically.
- ✓ Quick Watch lets us manually inspect a specific Variable or Expression.

The background of the slide is a photograph of a lecture hall. In the foreground, the silhouettes of several students are visible, some sitting and some standing, looking towards the front. A large, dark rectangular screen or whiteboard is positioned in the center of the room, serving as the focal point for the text. The overall lighting is dim, with the screen area being the brightest part of the image.

Introduction To Programming Using C++

Level 2

#06 Changing Values in Debugging Mode

Definition

- ◆ During debugging, we can inspect and sometimes modify the Values of Variables while the program is paused.
- ◆ This allows us to test different scenarios without changing the source code.

Explanation

- The debugger allows us to see the current state of Variables.
- In supported situations, we can change a Variable's Value while execution is paused.
- This can be useful for testing how the program behaves with different Values.
- Values can be modified through debugging windows such as the Autos Window or Watch Window.

Procedures

1. Start Debugging.
2. Stop the program at a Breakpoint.
3. Find the Variable in the Autos Window or Watch Window.
4. Edit the current Value.
5. Continue program execution.
6. Observe how the program behaves with the new Value.

💡 Example

- ◆ Suppose we have:
 `int age = 20;`
- ◆ The debugger is paused before a condition checks age.
- ◆ We can temporarily change age to 30.
- ◆ Then continue execution and observe how the program behaves.

➡ Result

We can test different Values and scenarios without modifying the source code.

Note

◆ Changes made during debugging are generally temporary and are not the same as changing the source code.

⊘ Warning

- ◆ Not every Variable or situation can be modified during debugging.
- ◆ Some Values may be read-only, optimized, unavailable, or otherwise restricted.

✦✦ Summary

- ✓ Debugging allows us to inspect Variable Values.
- ✓ We can sometimes modify Values while the program is paused.

The background of the slide is a photograph of a lecture hall. In the foreground, the silhouettes of several people are visible, looking towards a large screen. The screen displays a complex network of lines, possibly a diagram or a code visualization. The overall lighting is dim, with the screen being the primary light source.

Introduction To Programming Using C++

Level 2

#07 Step Into / Step Over Step Out

Definition

◆ Step Into, Step Over, and Step Out are debugging commands used to control how the debugger moves through the code.

Explanation

Step Into

-  Step Into executes the current statement and enters a called Function when possible.

-  It allows us to investigate the internal details of a Function.

-  It can move through Variable assignments, conditions, loops, and Function Calls step by step.

Step Over

-  Step Over executes the current statement.

-  If the statement contains a Function Call, the debugger executes the Function without entering its internal lines.

-  Execution then moves to the next statement in the current Function.

Step Out

-  Step Out is used when we are currently inside a Function.

-  It executes the remaining part of the current Function and returns to the calling code.

-  It is useful when we no longer need to inspect the internal details of the current Function.

Procedures

1. Use Step Into when you want to investigate inside a Function.
2. Use Step Over when you want to execute a Function Call without entering it.
3. Use Step Out when you are inside a Function and want to return to the calling Function.
4. Use F11 for Step Into.
5. Use F10 for Step Over.
6. Use Shift + F11 for Step Out.

💡 Example

◆ Suppose we have:

```
int result = CalculateSum(10, 20);
```

◆ If we use Step Into, the debugger can enter CalculateSum() and show its internal execution.

◆ If we use Step Over, CalculateSum() is executed as one step without entering its internal code.

◆ If we are already inside CalculateSum() and use Step Out, the debugger finishes the current Function and returns to the calling code.

➡ Result

We can control how deeply we move through the code during debugging..

Note

- ◆ Step Into → F11
- ◆ Step Over → F10
- ◆ Step Out → Shift + F11

⊘ Warning

- ◆ Step Into can take us into library or framework code that we may not need to inspect.
- ◆ In such cases, Step Over can make debugging easier.

Comparison

Step Into

- ✓ Goes inside a Function Call.
- ✓ Shows the internal execution of the Function.
- ✓ Useful when we need to understand what happens inside a Function.

Step Over

- ✓ Executes a Function Call without entering it.
- ✓ Keeps us in the current Function.
- ✓ Useful when the Function's internal implementation is not relevant.

Step Out

- ✓ Used when we are already inside a Function.
- ✓ Finishes the current Function and returns to the caller.
- ✓ Useful when we want to leave the current Function quickly.

✦✦ Summary

- ✓ Step Into → Go inside the Function.
- ✓ Step Over → Execute the Function without going inside it.
- ✓ Step Out → Leave the current Function and return to the caller.

Thank you 😊